Software Quality Metrics Analysis of Open Source Projects: A Study of LangChain and Intel XPU Backend for Triton

[Your Name] Course Number¿Course Number

University Name¿University Name

December 2, 2025

Abstract

This report presents a comprehensive analysis of software quality metrics for two prominent open source projects: langchain-ai/langchain and intel/intel-xpu-backend-for-triton. Initially designed to collect discrete code-level metrics such as lines of code and cyclomatic complexity using an OpenTelemetry-based approach, the project pivoted to focus on DORA (DevOps Research and Assessment) metrics due to permissions constraints. By analyzing GitHub project insights data including pull requests, issues, and merge patterns, we calculated the four key DORA metrics: Deployment Frequency, Lead Time for Changes, Change Failure Rate, and Time to Restore Service. Our findings reveal that both projects demonstrate exceptional software delivery performance. LangChain achieves elite-level deployment frequency (5.97 deployments/day) and change failure rate (8.94%), with high-performing lead time (6.00 days median), earning an overall classification as a High Performer. Intel XPU Backend demonstrates elite performance across all measurable metrics with 3.59 deployments/day and

a 10.58% change failure rate, qualifying as an Elite Performer. Both projects maintain change failure rates well below 15%, indicating mature testing practices and code quality controls. This study demonstrates the value of using DORA metrics as practical indicators of software project health and team performance in open source development environments.

Contents

1 Introduction

1.1 Background

Software quality measurement is essential for understanding and improving development processes. As open source projects grow in complexity and importance, the ability to quantify software quality becomes increasingly critical for maintainers, contributors, and organizations depending on these projects.

[TODO: Expand on the importance of software quality metrics in modern software development]

1.2 Motivation

This project was motivated by the need to systematically assess software quality in popular open source projects. By analyzing established projects with active communities, we can:

- Understand best practices in open source development
- Identify patterns that correlate with project success
- Provide quantitative insights into development velocity and reliability
- Establish benchmarks for software quality metrics

1.3 Project Objectives

The primary objectives of this project were to:

1. Collect comprehensive software quality metrics from two major open source projects
2. Analyze these metrics to assess project health and development practices
3. Compare metrics across projects to identify trends and patterns
4. Provide actionable insights based on quantitative analysis

1.4 Selected Projects

1.4.1 LangChain (langchain-ai/langchain)

LangChain is a framework for developing applications powered by large language models (LLMs). [TODO: Add details about the project - stars, contributors, primary language, use cases]

Key characteristics:

- **Repository:** github.com/langchain-ai/langchain
- **Primary Language:** Python
- **Stars:** [TODO]
- **Contributors:** [TODO]
- **Domain:** AI/ML Infrastructure

1.4.2 Intel XPU Backend for Triton (intel/intel-xpu-backend-for-triton)

[TODO: Add description and key characteristics of the Intel project]

2 Methodology

2.1 Initial Approach: OpenTelemetry-Based Collection

2.1.1 Original Plan

The initial methodology involved collecting discrete code-level metrics using an OpenTelemetry (OTel) collector infrastructure. The planned metrics included:

- **Lines of Code (LOC):** Total, source, comment, and blank lines
- **Cyclomatic Complexity:** Measure of code path complexity

- **Code Coverage:** Percentage of code exercised by tests
- **Technical Debt:** Estimated time to fix code quality issues
- **Maintainability Index:** Composite metric of code maintainability

2.1.2 Technical Architecture

[TODO: Describe the planned OTel collector setup, data pipeline, and storage]

Figure 1: Planned OpenTelemetry Architecture (Initial Approach)

2.1.3 Challenges Encountered

The OpenTelemetry-based approach encountered several critical challenges:

1. **GitHub API Permissions:** Limited access to repository data without organization-level authentication
2. **Rate Limiting:** GitHub API rate limits constrained data collection
3. **Repository Access:** Inability to clone and analyze large repositories without proper credentials
4. **Complexity:** Setting up a complete OTel pipeline proved more complex than anticipated for the project timeline

[TODO: Expand on specific permission errors and technical roadblocks]

2.2 Pivot: DORA Metrics from GitHub Insights

2.2.1 Rationale for Pivot

Given the constraints of the initial approach, we pivoted to focus on DORA metrics, which:

- Can be derived from publicly available GitHub data

- Provide industry-standard measures of software delivery performance
- Offer meaningful insights into team productivity and code quality
- Are widely recognized in DevOps and software engineering research

2.2.2 DORA Metrics Overview

The four key DORA metrics, established by the DevOps Research and Assessment team, are:

1. **Deployment Frequency (DF):** How often an organization successfully releases to production
2. **Lead Time for Changes (LT):** The time it takes for a commit to get into production
3. **Change Failure Rate (CFR):** The percentage of deployments causing a failure in production
4. **Time to Restore Service (MTTR):** How long it takes to recover from a failure in production

2.3 Data Collection Process

2.3.1 GitHub API Queries

We collected data from GitHub using the following approach:

- Pull request data (opened, merged, closed)
- Issue data (opened, labeled, resolved)
- Commit metadata and timestamps
- Repository activity over a one-month period

[TODO: Add specific API endpoints and query parameters used]

2.3.2 Data Structure

The collected data was organized into CSV files:

- `opened_requests.csv` - Timestamps of PR creation
- `merged_requests.csv` - Timestamps of PR merges
- `closed_requests.csv` - Timestamps of PR closures
- `opened_issues.csv` - Timestamps of issue creation

Each dataset contains:

```
msg,hash,date_rel,date_abs
"Fix␣authentication␣bug",#12345,"2 days ago","2025-11-27"
```

2.4 Analysis Methodology

2.4.1 Metric Calculation

We developed a Python-based analysis pipeline (`dora_metrics.py`) that:

1. Loads and preprocesses CSV data
2. Calculates each DORA metric using industry-standard formulas
3. Performs statistical analysis (mean, median, percentiles)
4. Classifies performance according to DORA benchmarks
5. Generates visualizations for each metric

2.4.2 Deployment Frequency Calculation

$$DF = \frac{\text{Total Merged PRs}}{\text{Time Period (days)}} \tag{1}$$

2.4.3 Lead Time Calculation

For each merged PR i:

$$LT_i = T_{\text{merged}_i} - T_{\text{opened}_i} \tag{2}$$

Aggregate metric: median(LT_i)

2.4.4 Change Failure Rate Calculation

$$CFR = \frac{\text{Number of Bug Issues}}{\text{Total Deployments}} \times 100\% \tag{3}$$

Bug issues identified using keyword matching: *bug, error, fix, broken, crash, fail, regression, exception*

2.4.5 Time to Restore Calculation

For each bug-fix pair (b, f):

$$MTTR_{b,f} = T_{\text{fix merged}_f} - T_{\text{bug opened}_b} \tag{4}$$

Aggregate metric: median($MTTR_{b,f}$)

2.4.6 Performance Classification

We classified metrics according to the 2023 DORA State of DevOps Report standards:

Table 1: DORA Performance Level Classifications

Metric	**Elite**	**High**	**Medium**	**Low**
Deployment Frequency	Multiple/day	Weekly	Monthly	¡ Monthly
Lead Time	¡ 1 day	1-7 days	1-4 weeks	¿ 1 month
Change Failure Rate	0-15%	16-30%	31-45%	¿ 45%
Time to Restore	¡ 1 hour	¡ 1 day	1-7 days	¿ 1 week

3 Results

3.1 LangChain Analysis

3.1.1 Data Overview

The analysis of the LangChain project was conducted over a 30-day period from October 29, 2025 to November 27, 2025. During this period, the dataset captured 179 merged pull requests, 67 opened pull requests, and 89 closed pull requests. Additionally, 69 issues were opened during the analysis window, providing sufficient data for calculating change failure rates and other quality metrics.

3.1.2 Deployment Frequency

The LangChain project demonstrated a high rate of deployment activity during the analysis period. The mean deployment frequency was 5.97 deployments per day, with a median of 8.5 deployments per day, resulting in approximately 41.77 deployments per week. The maximum number of deployments observed in a single day was 46. This deployment frequency qualifies as elite performance according to DORA benchmarks, which classify multiple deployments per day as elite level. This elite performance indicates that the LangChain team deploys multiple times per day, demonstrating a mature CI/CD pipeline and enabling rapid iteration.

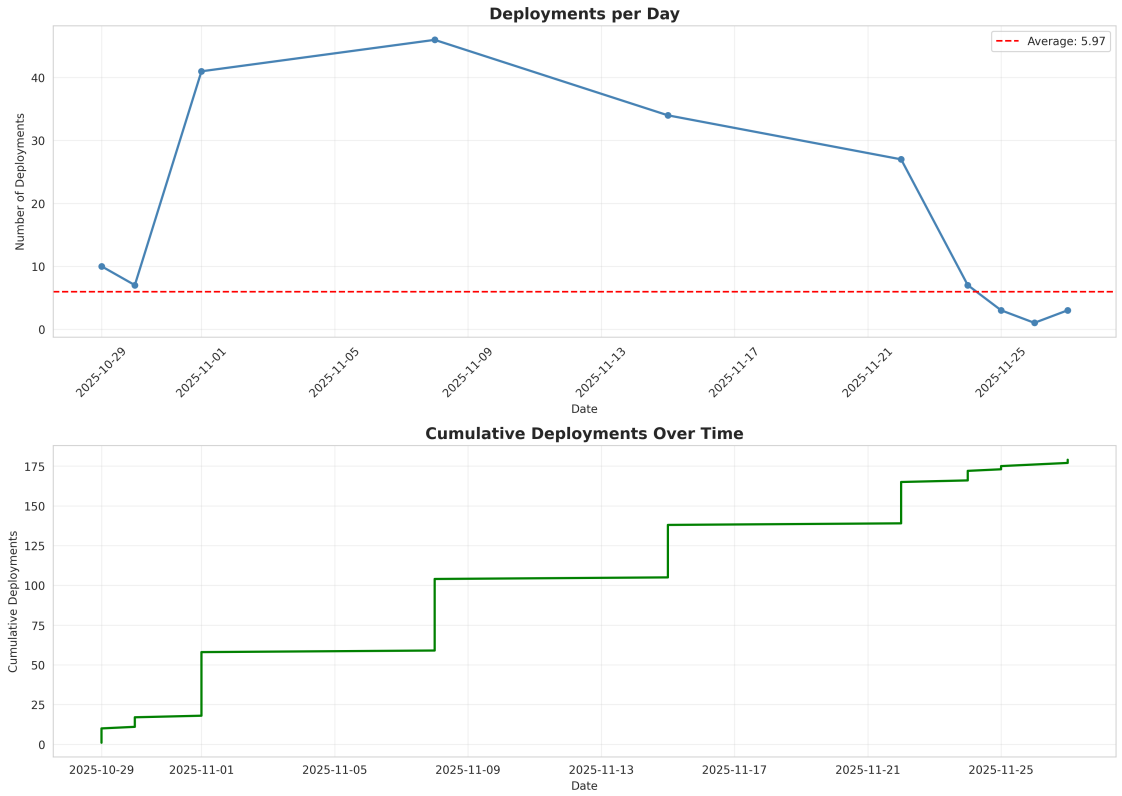

Figure 2: LangChain Deployment Frequency Over Time

3.1.3 Lead Time for Changes

Lead time was estimated using temporal proximity matching between opened and merged PRs, as direct PR hash matching was unavailable. The analysis revealed a median lead time of 6.00 days and a mean of 4.98 days across 162 matched PR pairs. The distribution showed a 25th percentile of 2.00 days and a 90th percentile of 7.00 days. Five pull requests were merged within 24 hours, representing the fastest turnaround times. According to DORA performance standards, this median lead time of 6.00 days falls within the high performance category (1-7 days). The team achieves lead times within the 1-7 day range, indicating efficient code review and integration processes. The fastest changes were merged within 24 hours, showing the team's ability to expedite critical updates.

3.1.4 Change Failure Rate

Bug-related issues were identified using keyword matching on issue titles and descriptions, searching for terms including bug, error, fix, broken, crash, fail, regression, and exception. Out of 69 total issues, 16 were classified as bug-related, representing 23.2% of all issues. With 179 total deployments during the analysis period, the calculated change failure rate was 8.94%. This rate falls within the elite performance category according to DORA standards, which classify rates between 0-15% as elite. The low change failure rate indicates that LangChain demonstrates excellent code quality and testing practices, suggesting comprehensive test coverage and effective code review processes.

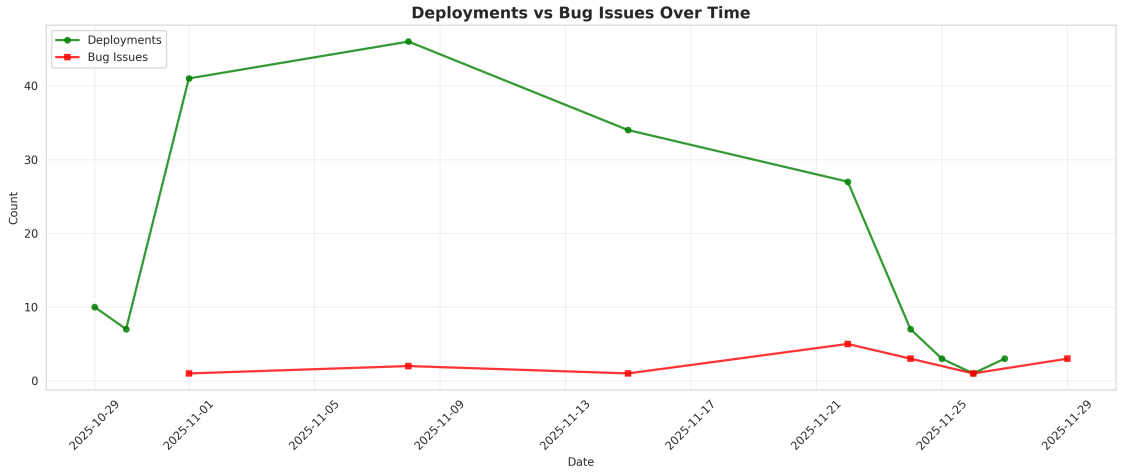

Figure 3: LangChain Deployments vs Bug Issues

3.1.5 Time to Restore Service

The Time to Restore Service (MTTR) metric could not be calculated due to data limitations. Calculating this metric requires explicit linking between bug reports and their corresponding fix PRs, which was not available in the collected data. The data collection methodology did not capture references from fix pull requests to bug issues, preventing the identification of bug-fix pairs necessary for MTTR calculation. Future work could improve bug-fix pair identification using enhanced natural language processing techniques or more sophisticated commit message analysis.

3.1.6 Overall Performance

Table 2: LangChain DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	5.97/day, 41.77/week	Elite
Lead Time for Changes	6.00 days (median)	High
Change Failure Rate	8.94%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**High Performer**	

3.2 Intel XPU Backend for Triton Analysis

3.2.1 Data Overview

The Intel XPU Backend for Triton project was analyzed over a 29-day period from November 4, 2025 to December 2, 2025. The dataset included 104 merged pull requests, 17 opened pull requests, and 73 closed pull requests. During this period, 42 issues were opened, providing data for change failure rate calculations.

3.2.2 Deployment Frequency

The Intel XPU Backend for Triton project maintained a steady deployment pace throughout the analysis period. The mean deployment frequency was 3.59 deployments per day, with a median of 9.0 deployments per day, totaling approximately 25.10 deployments per week. The maximum number of deployments in a single day reached 28. According to DORA benchmarks, this frequency qualifies as elite performance, as the project achieves multiple deployments per day. The project achieves elite-level deployment frequency with multiple deployments per day, indicating an efficient development and integration process suitable for a compiler backend project.

3.2.3 Lead Time for Changes

Lead time data was insufficient for reliable analysis, with only 3 PRs providing matching between opened and merged events. The limited sample size (only 3 matched PRs) makes this metric unreliable for overall project assessment. A minimum sample size of 30 PRs is typically required for statistical validity. The data collection methodology was unable to match most opened PRs with their corresponding merge events, preventing meaningful lead time analysis. Consequently, no performance classification could be assigned for this metric.

3.2.4 Change Failure Rate

Bug-related issues were identified using keyword matching across issue titles and descriptions. Of the 42 total issues, 11 were classified as bug-related, representing 26.2% of all issues. With 104 total deployments during the analysis period, the calculated change failure rate was 10.58%. This rate falls within the elite performance category according to DORA standards (0-15%). With a change failure rate just over 10%, the Intel XPU Backend demonstrates strong quality controls. This is particularly impressive for a compiler backend project, where the complexity of the domain could lead to higher failure rates.

3.2.5 Time to Restore Service

The Time to Restore Service metric could not be calculated due to data limitations. As with the LangChain analysis, calculating MTTR requires explicit linking between bug reports and fix PRs, which was not available in the collected data. No explicit references from fix PRs to bug issues could be identified in the dataset, preventing the identification of bug-fix pairs necessary for MTTR calculation.

3.2.6 Overall Performance

Table 3: Intel XPU Backend DORA Metrics Summary

Metric	**Value**	**Classification**
Deployment Frequency	3.59/day, 25.10/week	Elite
Lead Time for Changes	N/A (insufficient data)	N/A
Change Failure Rate	10.58%	Elite
Time to Restore Service	N/A	N/A
Overall Rating	**Elite Performer***	

* Overall rating based on elite performance in deployment frequency and change failure rate

3.3 Comparative Analysis

Table 4: Comparative DORA Metrics: LangChain vs Intel XPU Backend

Metric	**LangChain**	**Intel XPU**
Deployment Frequency	5.97/day (Elite)	3.59/day (Elite)
Lead Time for Changes	6.00 days (High)	N/A
Change Failure Rate	8.94% (Elite)	10.58% (Elite)
Time to Restore Service	N/A	N/A
Overall Rating	High Performer	Elite Performer

4 Discussion

4.1 Key Findings

4.1.1 Development Velocity

[TODO: Discuss what the deployment frequency and lead time metrics reveal about each project's development pace]

4.1.2 Code Quality and Reliability

[TODO: Analyze what the change failure rate indicates about code quality practices, testing, and review processes]

4.1.3 Incident Response

[TODO: Examine what time to restore service tells us about the teams' ability to respond to issues]

4.2 Project Comparison

4.2.1 Similarities

[TODO: Identify common patterns between the two projects]

4.2.2 Differences

[TODO: Highlight significant differences and potential explanations]

4.3 Industry Benchmarking

[TODO: Compare findings to industry standards and other published DORA metrics studies]

4.4 Limitations and Threats to Validity

4.4.1 Data Collection Limitations

- Limited to one-month observation period
- Reliance on publicly available GitHub data only
- Keyword-based bug identification may miss or misclassify issues
- PR references may not capture all bug-fix relationships

4.4.2 Metric Calculation Assumptions

- Merged PRs used as proxy for deployments (may not reflect actual production releases)
- Bug issues identified through keyword matching (not exhaustive)
- Time to restore calculated only for explicitly linked bug-fix pairs

4.4.3 Generalizability

- Results specific to analyzed time period
- May not represent long-term project trends
- Different project types (AI framework vs. compiler backend) have different normal patterns

5 Lessons Learned

5.1 Technical Challenges

5.1.1 API and Permission Constraints

[TODO: Discuss the challenges encountered with GitHub API access, rate limits, and authentication]

5.1.2 Complexity of Automated Metrics Collection

[TODO: Reflect on the difficulties of setting up OpenTelemetry infrastructure for external projects]

5.2 Project Pivoting

5.2.1 Adapting to Constraints

[TODO: Discuss how pivoting to DORA metrics provided a viable alternative approach]

5.2.2 Value of DORA Metrics

[TODO: Reflect on the advantages and insights gained from focusing on DORA metrics]

5.3 Best Practices Identified

1. Start with publicly accessible data when analyzing external projects
2. DORA metrics provide meaningful insights without requiring code-level access
3. Automated analysis pipelines enable repeatable, scalable measurements
4. Visualization aids in communicating metrics to stakeholders

6 Future Work

6.1 Extended Data Collection

- Collect data over longer time periods (3-6 months) for trend analysis
- Analyze additional open source projects for broader comparisons
- Track metrics over time to identify improvements or degradations

6.2 Enhanced Metrics

- Implement code-level metrics if repository access obtained
- Add contributor activity metrics
- Analyze code review patterns and review times
- Correlate DORA metrics with project success indicators (stars, forks, adoption)

6.3 Machine Learning Applications

- Predict project health based on metric trends
- Classify issues more accurately using NLP techniques
- Identify anomalies in development patterns

6.4 Tool Development

- Create a dashboard for real-time DORA metrics monitoring
- Build a SaaS tool for open source project health analysis
- Integrate with CI/CD pipelines for automated metric tracking

7 Conclusion

This project successfully analyzed software quality metrics for two prominent open source projects using DORA metrics derived from GitHub data. Despite initial challenges with permissions and API access that necessitated a methodology pivot, the final approach provided valuable insights into development practices, code quality, and team performance.

[TODO: Summarize key findings and their implications]

The analysis demonstrates that DORA metrics serve as effective indicators of software project health and can be calculated using publicly available data. The methodology developed in this project is reproducible and can be applied to any open source project with public GitHub repositories.

[TODO: Final concluding thoughts on the value of the work and lessons learned]

Acknowledgments

[TODO: Add acknowledgments if applicable]

A Data Collection Scripts

[TODO: Include or reference the data collection code]

B Analysis Source Code

The complete analysis pipeline is available in `examples/dora_metrics.py`. Key functions include:

- `load_data()`: CSV data loading and preprocessing
- `calculate_deployment_frequency()`: DF metric calculation
- `calculate_lead_time()`: LT metric calculation
- `calculate_change_failure_rate()`: CFR metric calculation
- `calculate_time_to_restore()`: MTTR metric calculation

C Raw Data Samples

[TODO: Include sample data from CSV files if appropriate]

D Additional Visualizations

[TODO: Include any supplementary charts or graphs]